CPUs vs GPUs

Summary

CPUs (a cheetah)	**GPUs** (an army of snails)
Evolved to run single-threaded code as fast as possible	Run massively parallel code
Nowadays : several cores (∼4-16) for additional parallelization	Must have 10K - millions of "work items" to use effectively
Optimize **latency**	Optimize **throughput**

Key idea

Learning goals

— Understand how GPU cores differ from CPU cores.

— Understand which workloads run better on GPUs vs CPUs.

Example

3D graphics, then and now

— Around 1996 : Nintendo 64, 320×400 pixels, 30 frames per second, runs at 62.5 MHz.

— 2025 consumer GPUs : AMD RX 7900 XTX (∼1K CHF), NVIDIA RTX 5090 (∼2K CHF, the card presented in detail below). Unreal Engine scenes.

Graphics is Brutal

Rendering a triangle : a small program (*shader*) computes the color of each pixel, e.g. load a texture value and multiply it by $\max(0, \texttt{dot(light_dir, normal)})$. Compiled to (hypothetical) machine instructions : `LOAD_TEXTURE`, `MUL`, `FMA`, `FMA`, `MAX`, `MUL`, `MUL`, `MUL`.

Key formulas

Modern requirements

— 4K resolution (3840 × 2160 pixels), 60 frames per second

— That's ∼**500 million** pixels per second…

  — × the number of instructions to produce a color (complex shaders have many more instructions)

  — × overdraw, multiple passes

— Millions of triangles in the scene

Bottom line : $\frac{1}{2}$ - $1\frac{1}{2}$ **trillion operations per second**. Normal processors can't keep up with that.

Definition

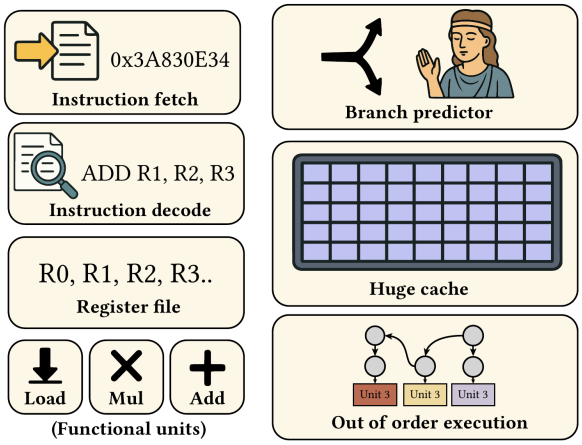
EMBARRASSINGLY PARALLEL PROBLEM
Rendering is one : let's compute all the pixels in parallel.

Warning

Problem : CPU-style processors are **too big**, we can only have a few. We will have to make some tough compromises…

From CPU to GPU

Review : CPU cores



Left : what we *want* to do (fetch, decode, register file, functional units). Right : what we *had* to do to make it fast (branch predictor, huge cache, out of order execution, prefetcher).

Key idea

Idea : throw away all parts intended to make single-threaded programs run fast.

Warning

Implications

— Without branch predictor : every jump is slow.

— Without out of order execution : an `ADD` waiting on a `LOAD` is slow.

— Without big, fast caches : everything is slow.

Key idea

When all pixels are computed in parallel, **all processors are evolving in lockstep** (same instructions on different data). Let's exploit this in the architecture.

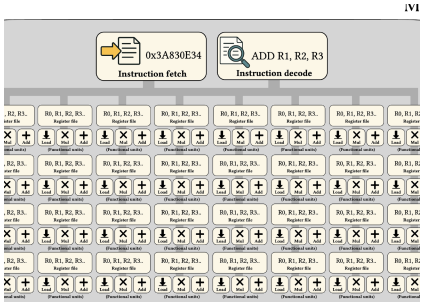
SM Partitions and Streaming Multiprocessors

Definition

SM PARTITION
32 tiny "processors" (each with its own register file and functional units : load, mul, add) **sharing one instruction fetch + decode**. Similar to vector operations on CPUs.

Definition

STREAMING MULTIPROCESSOR (SM)
4 SM partitions sharing an **L1 cache / shared memory** (128 KiB).



Key formulas

NVIDIA RTX 5090 (Blackwell GB202 chip)

$$170 \text{ SMs} \times 4 \text{ partitions} \times 32 \text{ processing elements} = 21'760 \text{ “cores”}$$

plus a **hardware scheduler**, an L2 cache (96 MiB) and the GPU memory (32 GiB).

Design Choices

Fast, complex circuit vs. slow, simple circuit : GPUs pick the second.

Key formulas

Latencies of different operations (approximate, worst case)

	RTX 5090	Intel Core Ultra 9 285K
Clock frequency	2.4 GHz	5.7 GHz
Load (L1 hit)	30 cycles	4 cycles
Load (L2 hit)	200 cycles	17 cycles
Load (Miss)	800 cycles	565 cycles

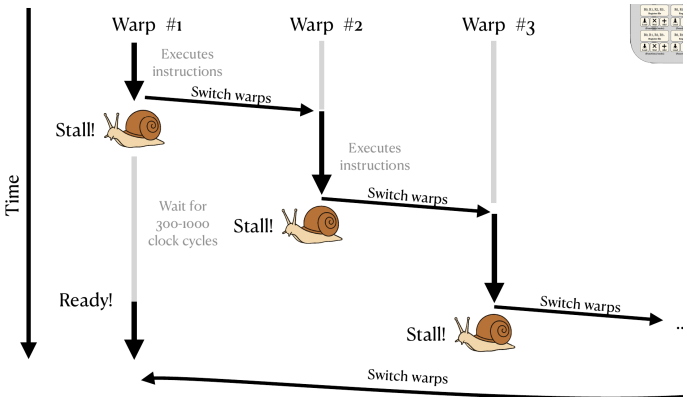
Key idea

Is that it ? Just a lot of small, slow processors ? No : two more ideas, latency hiding and shared memory.

Part 1 : Latency Hiding

Definition

WARP
A group of 32 threads, executed by an SM partition.



Key idea

- Because the cores on GPUs are so basic, they will **stall all the time** ; latencies from stalls are *terrible* for performance.
- Stalls are unavoidable, but GPUs can usually “hide” this latency.
- Key idea : run **many** warps per SM partition (**huge register file**).
  - Switch to another warp whenever a stall happens (a stalled warp waits for 300-1000 clock cycles).
  - Switching is “for free” (consumes no clock cycles).
  - If all goes well : do useful work at every clock cycle.
- Follows the high level pattern : optimize **throughput**, not latency.

Part 2 : Shared Memory

Warning

Most programs read and write a lot of memory : on GPUs, this means that they will stall constantly.

Definition

SHARED MEMORY (solution)
- Copy a small part of the problem into local (“shared”) memory.
- Do **a lot of work** with this copy, which is fast to access.
- Then write back the result.

GPUs as Compute Accelerators

Definition

KERNEL
You cannot run a browser or an operating system on a GPU : it merely helps to accelerate tasks. Its basic computational unit is called a **kernel** (the CPU asks : “Please run kernel `matmul_fp16_1024`”, the GPU : “OK!”).

Definition

KERNEL LAUNCH
Code + a **launch grid** : the grid specifies how many instances of the program to launch, e.g. $128 \times 128 \times 32$ times.

Example

GPUs for general purpose computing
- Originally, GPUs could only render triangles. Main use : computer games !
- People soon realized that their architecture is *interesting* for non-graphics usage.
- Hacky workarounds : full-screen rectangle + input data (as a texture) + graphics program = output “image” (*Brook for GPUs : Stream Computing on Graphics Hardware*, Buck et al. 2004).
- Today’s frameworks make GPU programming easier : **CUDA** (NVIDIA), **ROCm** (AMD), **SYCL** (Intel).

AI and Tensor Cores

Example

Developments in the last 5 years : NVIDIA's stock price went up 1'202.17%, currently the most valuable company on earth (market capitalization). Not because of computer games : datacenter GPUs (∼50'000 CHF each) for **artificial intelligence**. GPUs are really, really good at **matrix multiplication**, the fundamental operation underlying AI.

Definition

TENSOR CORE
Fancy name for a **matrix multiplication unit**. Like an FMA (fused-multiply-add) machine instruction :

— FMA : `D = A * B + C`

— Tensor core : `D = A @ B + C`

Multiplies a fixed matrix size quickly (e.g. 256×32 @ 32×256).

Key idea

— Nowadays most of the computational "power" of a GPU is in the tensor core.

— Recent trend : very low precision floating point (16 bit $\rightarrow$ 8 bit $\rightarrow$ 4 bit "floats").

Key formulas

Percentage of FLOP/s in tensor core :

	16 bit	8 bit	4 bit
B200 (data center)	98.4%	99.2%	99.6%
RTX 5090	75.0%	87.49%	93.8%

Tensor Core on Consumer GPUs : HMMA.16816

Definition

HMMA.16816
$16 \times 8 \times 16$ half precision matrix multiplication : **256 flops/cycle per SM partition**. Reads 768 bytes of input registers, keeps the tensor core busy for 16 cycles (4096 FLOPs), writes 256 bytes of output registers.

Key formulas

Some data for Ada A6000

— **64 TB/s bandwidth** needed just for inputs!

— Global memory : 960 GB/s (ideal)

— L2 cache : 5 TB/s (measured)

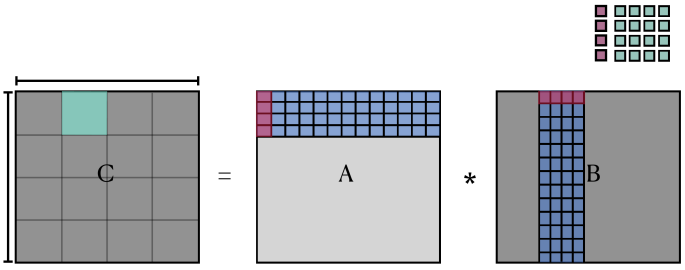
— Shared memory : 45.5 TB/s (ideal)

Warning

Not even shared memory is fast enough. How can one even use this operation ?

Key idea

Block matrix multiplication : each block of C is computed from a band of blocks of A and of B (16×16 blocks for HMMA.16816), loaded into shared memory and reused for many products.



Weekly Summary

Summary

— Key property of matrix multiplication : $O(n^3)$ **compute for** $O(n^2)$ **data**.

  — Exploit this by fetching parts of a matrix into local ("shared") memory.

  — Then do a lot of computation with these parts.

  — Allows matrix computation to be **compute-bound** instead of **memory-bound**.

— GPUs are much better at multiplying big matrices than CPUs.

— Matrix multiplication is the key operation underlying ML / AI (in particular, large language models use **huge** matrices).